Simple Fortran Derived Types and Python

Rohit Goswami · 2021-10-02 · python · fortran · numpy · f2py · ramblings

> *Moving simple Fortran derived types to Python and back via C*

Background

Object oriented programming has been part of Fortran for longer than I have been alive [^fn:1]. Fortran has derived types now. They've been around for around for over three decades. The standards at the same time, have been supporting more and more interoperable operations. Details of these pleasant historical improvements are pretty much the most the Fortran standards committee have managed to date in the 21st century. The point of this exploration is to design interfaces to `Python`, mostly for the long term advancement of `f2py`, and just for the heck of it. Simple in the context of the title means that we consider only derived types whose members have direct `C` equivalents as defined in the Fortran 2003 Reid (2003) standard.

C Structures and Derived Types - I

Consider, first an example of what shall be a rather simple collection of records, the prototypical darling of most simulations, the point in Euclidean space, $\mathbb{R}\^3$.

```
type, bind(c) :: cartesian
   real(c_double) :: x,y,z
end type cartesian
```

Now by definition, ever since the `ISO_C_BINDING` of the Fortran 2003 Reid (2003) standard, we know this is equivalent to:

```
typedef struct {
  double x,y,z;
} cartesian;
```

To try this out, we need to have a procedure which operates on the type, due to a chronic lack of imagination, let us simply take a point and add one to each axis.

```
subroutine unit_move(cartpoint) bind(c)
  type(cartesian), intent(inout) :: cartpoint
  print*, "Modifying the derived type now!"
  cartpoint%x=cartpoint%x+1
  cartpoint%y=cartpoint%y+1
  cartpoint%z=cartpoint%z+1
end subroutine unit_move
```

Now we can wrap these into a nice little module:

```
module vec
  use, intrinsic :: iso_c_binding
  implicit none

  type, bind(c) :: cartesian
     real(c_double) :: x,y,z
  end type cartesian

  contains

  subroutine unit_move(array) bind(c)
    type(cartesian), intent(inout) :: array
    print*, "Modifying the derived type now!"
    array%x=array%x+1
    array%y=array%y+1
    array%z=array%z+1
  end subroutine unit_move

end module vec
```

Call it with a simple `main`.

```
! main.f90
program main
  use, intrinsic :: iso_c_binding
  use vec
  implicit none
  type(cartesian) :: cartvec
  cartvec = cartesian(0.2,0.5,0.3)
  print*, cartvec
  call unit_move(cartvec)
  print*, cartvec
end program main
```

Finally we need a build system, lets say `meson`.

```
# meson.build
project('fcbase', 'fortran',
  version : '0.1',
  default_options : ['warning_level=3'])

executable('fcbase',
           'vec.f90',
           'main.f90',
           install : true)
```

Now we can finally try this out.

```
meson setup testfc
meson compile -C testfc
./testfc/fcbase
  0.20000000298023224       0.50000000000000000       0.30000001192092896
 Modifying the derived type now!
   1.2000000029802322        1.5000000000000000        1.3000000119209290
```

This probably surprised no one. So let's get to something more interesting.

C and Fortran

We can start with a simple `C` header.

```
#ifndef VECFC_H
#define VECFC_H

typedef struct {
  double x,y,z;
} cartesian;

#endif /* VECFC_H */
```

Along with its associated translation unit.

```
/* vecfc.c */
#include<stdlib.h>
#include<stdio.h>
#include<vecfc.h>

void* unit_move(cartesian *word);

int main(int argc, char* argv[argc+1]) {
    puts("Initializing the struct");
    cartesian a={3.0, 2.5, 6.2};
    printf("%f %f %f",a.x,a.y,a.z);
    puts("\nFortran function with derived type from C:");
    unit_move(&a);
    puts("\nReturned from Fortran");
    printf("%f %f %f",a.x,a.y,a.z);
    return EXIT_SUCCESS;
}
```

The function declaration is straightforward, but for the fact that since we are passing something both in and out of the function, it must be a pointer, and as we have no real error propagation or feasible return type, `void *` makes sense too. The calling semantics are equally transparent, creating an instance of the `struct` and then passing it by reference.

The meson extension is fairly trivial, we only need to add the right file and declare the project language correctly.

```
project('cderived', 'c', 'fortran',
  version : '0.1',
  default_options : ['warning_level=3'])

executable('cderived',
           'vec.f90',
           # 'vecfc.c',
           # 'vecfc.h',
           'main.f90',
           install : true)
```

This seems to work.

```
meson setup testfc
meson compile -C testfc
./testfc/fcbase
Initializing the struct
3.000000 2.500000 6.200000
Fortran function with derived type from C:
 Modifying the derived type now!

Returned from Fortran
4.000000 3.500000 7.200000%
```

Cython and Fortran

So far so good. Let's get to something slightly more interesting. Although in the long run `cython` isn't really the approach `numpy` is going towards at the moment, it still makes sense to give it a shot as a first approximation.

```
cdef struct cartesian:
    double x,y,z

cdef extern:
    void* unit_move(cartesian *word)

def unitmv(dpt={'x':0,'y':0,'z':0}):
    cdef cartesian A=dpt
    unit_move(&A)
    return(A)
```

The most interesting aspect is that, having defined our `C` structure and the external function, we need something to offer to `Python`. Since `cython` structures can be mapped from simple `python` dictionaries, we take an input of one, construct the `structure` on the fly in the function and then use our Fortran subroutine. The return type can also be coerced into a list, and that's what can be printed or manipulated further. This isn't the most efficient way to work with this, but it will do for now.

The `meson` update follows from the explanation in the previous post, namely, that we need to include our `Python` sources.

```
project('pointeuc', 'c', 'fortran',
  version : '0.1',
  default_options : ['warning_level=3'])

add_languages('cython')

py_mod = import('python')
py3 = py_mod.find_installation()
py3_dep = py3.dependency()

incdir_numpy = run_command(py3,
  ['-c', 'import os; os.chdir(".."); import numpy; print(numpy.get_include())'],
  check : true
).stdout().strip()

inc_np = include_directories(incdir_numpy)

py3.extension_module('veccyf',
                     'veccyf.pyx',
                     'vec.f90',
                     include_directories: inc_np,
                     dependencies : py3_dep)
```

The `NumPy` dependency in this situation is really optional, but makes for a more interoperable structure. Now we're ready to try this out in an interactive manner.

```
meson setup testcpyth
meson compile -C testcpyth
cd testcpyth
python -c "import veccyf; A=veccyf.unitmv({'x':3.3,'y':5.2,'z':2.6}); print(A)"
 Modifying the derived type now!
{'x': 4.3, 'y': 6.2, 'z': 3.6}
```

Taking this a bit slower, it is still as neat as ever.

```
import veccyf
point = {'x': 4.3,
         'y': 2.6,
         'z': 6.4}
opoint = veccyf.unitmv(point)
print(opoint)
```

Aside from the output string, this works as expected, and is pretty neat, all things considered. This might feed into an `f2py` workflow, but then, it adds a layer of indirection which seems overly circuitous. The design could have also been far improved by using a `bind(c, name=uniquname)` as well.

Note that a much more rational approach would have been to generate a class in Python.

Prognosis for F2PY

The essential information used from the `fortran` code for the `cython` types (and `C` structures) was essentially:

- The types of each variable for the derived type
  - `double` for every one of them in this case
- The names of each variable
  - `x`, `y`, and `z` here

Apart from this, for functions which operate on aforementioned types,we require:

- The external declaration (due to the `iso_c_binding`)
- A mapping from structure to a python object, typically a dictionary

It so happens that much of the information needed for generation of the relevant `C` code is already present in `f2py`, and can be inspected via the `crackfortran` module.

```
# python -m numpy.f2py.crackfortran vec.f90 -show
Reading fortran codes...
        Reading file 'vec.f90' (format:free)
{'before': '', 'this': 'use', 'after': ', intrinsic :: iso_c_binding '}
Line #2 in vec.f90:"  use, intrinsic :: iso_c_binding "
        analyzeline: Could not crack the use statement.
Line #5 in vec.f90:"  type, bind(c) :: cartesian "
        analyzeline: No name/args pattern found for line.
Post-processing...
        Block: vec
                Block: unknown_type
                Block: unit_move
Post-processing (stage 2)...
[{'args': [],
  'block': 'module',
  'body': [{'args': [],
            'block': 'type',
            'body': [],
            'entry': {},
            'externals': [],
            'from': 'vec.f90:vec',
            'interfaced': [],
            'name': 'unknown_type',
            'parent_block': <Recursion on dict with id=4500228096>,
            'sortvars': ['x', 'y', 'z'],
            'varnames': ['x', 'y', 'z'],
            'vars': {'x': {'kindselector': {'kind': 'c_double'},
                           'typespec': 'real'},
                     'y': {'kindselector': {'kind': 'c_double'},
                           'typespec': 'real'},
                     'z': {'kindselector': {'kind': 'c_double'},
                           'typespec': 'real'}}},
           {'args': ['array'],
            'block': 'subroutine',
            'body': [],
            'entry': {},
            'externals': [],
            'from': 'vec.f90:vec',
            'interfaced': [],
            'name': 'unit_move',
            'parent_block': <Recursion on dict with id=4500228096>,
            'saved_interface': '\n'
                               '      subroutine unit_move(array) \n'
                               '          type(cartesian), intent(inout) :: '
                               'array\n'
                               '      end subroutine unit_move',
            'sortvars': ['array'],
            'vars': {'array': {'attrspec': [],
                               'intent': ['inout'],
                               'typename': 'cartesian',
                               'typespec': 'type'}}}],
  'entry': {},
  'externals': [],
  'from': 'vec.f90',
  'implicit': None,
  'interfaced': [],
  'name': 'vec',
  'sortvars': [],
  'vars': {}}]
```

The information is clearly present, which brings us to the next phase.

Python-C and Fortran

The most natural way to feed into `f2py` would be to directly write out an equivalent `Python-C` API which effectively mimics the extension module of the last section.

The critical definition essentially takes a dictionary, unrolls into a list, creates the `C` structure, then passes it through Fortran and then back.

```
static PyObject* eucli_unitinc(PyObject* self, PyObject* args) {
    cartesian a;
    PyObject* dict;
    PyArg_ParseTuple(args, "O", &dict);
    PyObject* vals = PyDict_Values(dict);
    a.x = PyFloat_AsDouble(PyList_GetItem(vals,0));
    a.y = PyFloat_AsDouble(PyList_GetItem(vals,1));
    a.z = PyFloat_AsDouble(PyList_GetItem(vals,2));
    // Call Fortran on it
    unit_move(&a);
    //
    PyObject* ret = Py_BuildValue("{s:f,s:f,s:f}",
                        "x", a.x,
                        "y", a.y,
                        "z", a.z);
    return ret;
}
```

This also works well enough with minor modifications to `meson`.

```
python -c "import eucli; A=eucli.unitinc({'x':3.3,'y':5.2,'z':2.6}); print(A)"
 Modifying the derived type now!
{'x': 4.3, 'y': 6.2, 'z': 3.6}
```

Boilerplate

The rest of the hand written wrapper is boilerplate, but the full file is reproduced here:

```
/* Make "s#" use Py_ssize_t rather than int. */
#ifndef PY_SSIZE_T_CLEAN
#define PY_SSIZE_T_CLEAN
#include <stdio.h>
#endif /* PY_SSIZE_T_CLEAN */

#include "Python.h"

// Fortran-C stuff

typedef struct {
  double x,y,z;
} cartesian;

void* unit_move(cartesian *word);

// Python-C stuff

static PyObject* eucli_unitinc(PyObject* self, PyObject* args) {
    cartesian a;
    PyObject* dict;
    PyArg_ParseTuple(args, "O", &dict);
    PyObject* vals = PyDict_Values(dict);
    a.x = PyFloat_AsDouble(PyList_GetItem(vals,0));
    a.y = PyFloat_AsDouble(PyList_GetItem(vals,1));
    a.z = PyFloat_AsDouble(PyList_GetItem(vals,2));
    // Call Fortran on it
    unit_move(&a);
    //
    PyObject* ret = Py_BuildValue("{s:f,s:f,s:f}",
                         "x", a.x,
                         "y", a.y,
                         "z", a.z);
    return ret;
}

static PyMethodDef EucliMethods[] = {
    {"unitinc", (PyCFunction)eucli_unitinc, METH_VARARGS,
     "Add 1 to the axes of a point"},
    {NULL, NULL, 0, NULL}        /* Sentinel */
};

static struct PyModuleDef eucli_module = {
   PyModuleDef_HEAD_INIT,
   "eucli",
   "blah",
   -1,
   EucliMethods
};

PyMODINIT_FUNC PyInit_eucli(void)
{
    PyObject *m;
    m = PyModule_Create(&eucli_module);
    if (!m) {
        return NULL;
    }

    return m;
}
```

Conclusions

Slowly but surely, compilers have caught up with to the standards, and soon, with some support, `numpy` shall too. A subsequent post shall cover more exotic derived types, typically (until 2018) treated by non-portable opaque pointer tricks. There are still some rather awkward design decisions in this case; including things like considering the mapping and their interoperability with `NumPy` arrays, but by and large this would depend on the functions which use these. In terms of derived types of simple structures, without allocatable arrays, the extensions appear to be straightforward.

References

Reid, John. 2003. "The New Features of Fortran 2003," 38. <https://wg5-fortran.org/N1551-N1600/N1579.pdf>.